REPORT OF SCHEMA AND RELATIONSHIPS

Name:	Tanvi Aggarwal
Project Title:	Flipkart Sales Analysis
Duration:	26th June 2025 – 29th June 2025
Focus:	System Design & Relational Database Development
Domain:	E-Commerce Industry
Tool Used:	MySQL Workbench
Language:	SQL

TABLE OF CONTENT

1. Introduction

2. Schema Overview

3. Tables Description
    - Categories
    - Customers
    - Orders
    - OrderDetails
    - Products
    - Returns
    - Suppliers

4. Table Relationships

5. SQL Data Analysis Questions
    - Basic Data Retrieval
    - Joins and Aggregates
    - Conditional Logic (CASE)
    - Window Functions
    - CTEs (Common Table Expressions)
    - Views
    - Subqueries and Advanced Filtering

6. Conclusion

INTRODUCTION

The Flipkart Sales Analysis project is a data analytics initiative aimed at uncovering valuable business insights from Flipkart's sales data. With the growing importance of data in driving decision-making, this project focuses on analyzing key performance indicators (KPIs) such as product sales, customer activity, return rates, and regional performance using MySQL.

This analysis is performed on a structured dataset that mimics Flipkart's e-commerce operations, consisting of multiple relational tables—customers, orders, order details, products, categories, suppliers, and returns. These tables represent the various facets of an online retail platform, offering a holistic view of business operations from order placement to product returns.

The primary objective is to derive actionable insights that can support decision-making in areas such as inventory management, marketing strategy, pricing optimization, and customer engagement. For example, identifying top-selling categories, analyzing the return rate of specific products, and spotting loyal or high-value customers can help Flipkart fine-tune its strategies.

By writing and executing 70+ SQL queries across diverse analytical dimensions—such as joins, aggregations, CTEs, window functions, views, and subqueries—this project demonstrates how data can be transformed into strategic knowledge. The analysis not only supports day-to-day operations but also helps in long-term planning by highlighting patterns and trends across time and regions.

In short, this project bridges technical SQL skills with business understanding, showcasing how structured data analysis can contribute to smarter, data-driven e-commerce decisions.

SCHEMA OVERVIEW

1. **Customers**

Stores information about Flipkart customers.

- CustomerID, CustomerName, Email, City, Region, JoinDate

2. **Categories**

Defines the different product categories available on the platform.

- CategoryID, CategoryName

3. **Suppliers**

Contains data about suppliers responsible for providing the listed products.

- SupplierID, SupplierName, Region

4. **Products**

Stores product-related information including pricing and linkage to categories and suppliers.

- ProductID, ProductName, CategoryID, SupplierID, Price

5. **Orders**

Captures customer orders and related delivery and payment information.

- OrderID, CustomerID, OrderDate, ShippingCity, EstimatedDeliveryDate, DeliveryDate, Status, PaymentMethod

6. **OrderDetails**

Records each product included in an order along with quantity and pricing details.

- OrderDetailID, OrderID, ProductID, Quantity, SellingPrice

7. **Returns**

- Tracks returned orders and the reasons for return.
  ReturnID, OrderID, ReturnDate, Reason

TABLES

- **Customers**

Column Name	Data Type	Constraints
CustomerID	INT	PRIMARY KEY
CustomerName	VARCHAR(100)	NOT NULL
Email	VARCHAR(100)	UNIQUE, NULL
City	VARCHAR(100)	NULL
Region	VARCHAR(50)	NULL
JoinDate	DATE	NULL

- **Categories**

Column Name	Data Type	Constraints
CategoryID	INT	PRIMARY KEY
CategoryName	VARCHAR(100)	NOT NULL

- **Suppliers**

Column Name	Data Type	Constraints
SupplierID	INT	PRIMARY KEY,
SupplierName	VARCHAR(150)	NOT NULL
Region	VARCHAR(50)	NOT NULL

- **Products**

Column Name	Data Type	Constraints
ProductID	INT	PRIMARY KEY
ProductName	VARCHAR(100)	NOT NULL
CategoryID	INT	FOREIGN KEY REFERENCES Categories(CategoryID)

SupplierID	INT	FOREIGN KEY REFERENCES Suppliers(SupplierID)
Price	DECIMAL(10,2)	NOT NULL

- **Orders**

Column Name	**Data Type**	**Constraints**
OrderID	INT	PRIMARY KEY
CustomerID	INT	FOREIGN KEY REFERENCES Customers(CustomerID)
OrderDate	DATE	NOT NULL
ShippingCity	VARCHAR(100)	NOT NULL
EstimatedDeliveryDate	DATE	NOT NULL
DeliveryDate	DATE	NOT NULL
Status	VARCHAR(50)	NOT NULL
PaymentMethod	VARCHAR(50)	NOT NULL

- **OrderDetails**

Column Name	**Data Type**	**Constraints**
OrderDetailID	INT	PRIMARY KEY
OrderID	INT	FOREIGN KEY REFERENCES Orders(OrderID)

ProductID	INT	FOREIGN KEY REFERENCES Products(ProductID)
Quantity	INT	NOT NULL
SellingPrice	DECIMAL(10,2)	NOT NULL
DeliveryDate	DATE	NOT NULL

- **Returns**

Column Name	**Data Type**	**Constraints**
ReturnID	INT	PRIMARY KEY
OrderID	INT	FOREIGN KEY REFERENCES Orders(OrderID)
ReturnDate	DATE	NOT NULL
Reason	VARCHAR(100)	NOT NULL

RELATIONSHIPS

The **FlipkartSales** database schema has been designed using a relational model to reflect real-world business operations such as customer purchases, product listings, returns, and supplier relationships. Each table is linked through **primary keys** and **foreign keys**, ensuring data integrity and enabling advanced analysis through SQL joins.
Below is an in-depth look at the key relationships between the seven tables:

🔹 1. Customers – Orders (One-to-Many)

Each customer can place multiple orders, but each order is associated with one and only one customer.

- **Primary Key:** CustomerID in Customers
- **Foreign Key:** CustomerID in Orders

This relationship allows us to:

- Track all orders placed by a particular customer
- Analyze customer-specific purchase behavior
- Join with other tables (like OrderDetails, Products) for customer-level analysis

🔹 2. Orders – OrderDetails (One-to-Many)

Every order can contain multiple products, and each entry in OrderDetails refers to one product in one order.

- **Primary Key:** OrderID in Orders
- **Foreign Key:** OrderID in OrderDetails

This relationship supports:

- Detailed transaction-level analysis
- Tracking of how many products were included in each order
- Revenue calculations at a granular level

🔹 3. OrderDetails – Products (Many-to-One)

Each record in OrderDetails corresponds to a single product, but a product may appear in many orders.

- **Primary Key:** ProductID in Products
- **Foreign Key:** ProductID in OrderDetails

Uses include:

- Sales trend analysis of individual products
- Identification of top-selling or frequently returned products
- Revenue contribution of products over time

🔹 4. Products – Categories (Many-to-One)

Every product belongs to one category, but a category can contain multiple products.

- **Primary Key:** CategoryID in Categories
- **Foreign Key:** CategoryID in Products

This enables:

- Category-wise sales aggregation
- Comparison of performance across categories
- Understanding consumer preference by category

🔹 5. Products – Suppliers (Many-to-One)

Each product is supplied by one supplier, but a supplier may provide many products.

- **Primary Key:** SupplierID in Suppliers
- **Foreign Key:** SupplierID in Products

Through this relationship, we can:

- Evaluate supplier-wise product performance
- Track region-wise sourcing trends
- Assess dependence on key suppliers

🔹 6. Orders – Returns (One-to-One or One-to-Many)

Some orders may be returned, with one or multiple return entries (depending on system design). Each return links to one order.

- **Primary Key:** OrderID in Orders
- **Foreign Key:** OrderID in Returns

This supports:

- Return rate analysis
- Identifying patterns in returns by category, supplier, or region
- Root cause analysis based on return reasons

🔹 7. Overall Join Flow (Across Tables)

Thanks to these foreign key relationships, we can create advanced SQL queries that:

- Join Customers → Orders → OrderDetails → Products → Categories/Suppliers
- Analyze delivery delays (Orders), total revenue (OrderDetails), and refund behavior (Returns)
- Generate customer-specific or product-specific dashboards

SQL DATA ANALYSIS

➢ SECTION 1:

1.

```
1   -- 1. Retrieve all orders that have been marked as 'Delivered'
2   USE FlipkartSales;
3   SELECT * FROM Orders WHERE Status = 'Delivered';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

OrderID	CustomerID	OrderDate	ShippingCity	EstimatedDeliveryDate	DeliveryDate	Status	PaymentMethod
1	20	2024-03-23	East Josephville	2024-05-10	2024-05-08	Delivered	Cash
4	9	2024-01-24	Port Shannon	2024-01-28	2024-01-31	Delivered	Cash
5	4	2024-04-15	Christinaton	2024-05-20	2024-05-22	Delivered	UPI
7	12	2024-05-08	Lake Mark	2024-05-13	2024-05-14	Delivered	Cash
8	5	2024-02-28	East Tonya	2024-03-05	2024-03-04	Delivered	NetBanking
9	1	2024-01-13	Johnstonville	2024-01-18	2024-01-19	Delivered	UPI
11	12	2023-06-15	Mumbai	2023-06-20	2023-06-20	Delivered	UPI
13	8	2023-06-17	Bangalore	2023-06-22	2023-06-21	Delivered	Cash
15	33	2023-06-19	Chennai	2023-06-24	2023-06-25	Delivered	UPI
16	18	2023-06-20	Kolkata	2023-06-25	2023-06-26	Delivered	Credit Card
18	9	2023-06-22	Jaipur	2023-06-27	2023-06-26	Delivered	Net Banking
19	51	2023-06-23	Ahmedabad	2023-06-28	2023-06-28	Delivered	Credit Card
21	63	2023-06-25	Lucknow	2023-06-30	2023-06-29	Delivered	UPI
23	74	2023-06-27	Indore	2023-07-02	2023-07-02	Delivered	Net Banking
25	39	2023-06-29	Patna	2023-07-04	2023-07-03	Delivered	UPI
26	71	2023-06-30	Vadodara	2023-07-05	2023-07-05	Delivered	Credit Card
27	50	2023-07-01	Ranchi	2023-07-06	2023-07-06	Delivered	Cash
29	60	2023-07-03	Nashik	2023-07-08	2023-07-08	Delivered	UPI
31	80	2023-07-05	Chandigarh	2023-07-10	2023-07-09	Delivered	UPI
32	43	2023-07-06	Amritsar	2023-07-11	2023-07-10	Delivered	Cash
34	17	2023-07-08	Jabalpur	2023-07-13	2023-07-12	Delivered	Debit Card
36	65	2023-07-10	Meerut	2023-07-15	2023-07-15	Delivered	UPI
37	24	2023-07-11	Rajkot	2023-07-16	2023-07-16	Delivered	Net Banking
39	7	2023-07-13	Noida	2023-07-18	2023-07-18	Delivered	UPI
41	44	2023-07-15	Kochi	2023-07-20	2023-07-20	Delivered	UPI
43	88	2023-07-17	Ludhiana	2023-07-22	2023-07-22	Delivered	Net Banking
45	23	2023-07-19	Jodhpur	2023-07-24	2023-07-25	Delivered	Cash
46	69	2023-07-20	Amravati	2023-07-25	2023-07-26	Delivered	UPI
48	2	2023-07-22	Vellore	2023-07-27	2023-07-28	Delivered	Credit Card
49	58	2023-07-23	Aurangabad	2023-07-28	2023-07-29	Delivered	Debit Card
51	27	2023-07-25	Hubli	2023-07-30	2023-07-30	Delivered	UPI
53	38	2023-07-27	Tirupati	2023-08-01	2023-08-01	Delivered	Net Banking
55	66	2023-07-29	Gorakhpur	2023-08-03	2023-08-03	Delivered	Cash
56	20	2023-07-30	Bilaspur	2023-08-04	2023-08-05	Delivered	UPI
57	41	2023-07-31	Ujjain	2023-08-05	2023-08-06	Delivered	Credit Card

2.

```sql
-- 2. List all orders with a total selling price greater than ₹5,000
USE FlipkartSales;
SELECT OrderID
FROM OrderDetails
GROUP BY OrderID
HAVING SUM(SellingPrice * Quantity) > 5000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

OrderID
1
2
3
4
5
6
7
14
31
33
34
35
44
45
47
63
64
75
77
78

3.

```sql
-- 3. Find the total number of orders per city
USE FlipkartSales;
SELECT ShippingCity, COUNT(*) AS TotalOrders
FROM Orders
GROUP BY ShippingCity;
```

Result Grid | Filter Rows: | Export: | Wrap C

ShippingCity	TotalOrders
East Josephville	1
West Juan	1
Lake Matthew	1
Port Shannon	1
Christinaton	1

4.

```sql
-- 4. Show all customers who have placed more than 5 orders
USE FlipkartSales;
SELECT CustomerID, COUNT(*) AS OrderCount
FROM Orders
GROUP BY CustomerID
HAVING COUNT(*) > 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	OrderCount

5.

```sql
-- 5. List all categories where total quantity sold exceeds 1,000 units
USE FlipkartSales;
SELECT c.CategoryName, SUM(od.Quantity) AS TotalQuantity
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
JOIN Categories c ON p.CategoryID = c.CategoryID
GROUP BY c.CategoryID
HAVING SUM(od.Quantity) > 1000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CategoryName	TotalQuantity

6.

```sql
-- 6. Display all products with a return rate greater than 10%
USE FlipkartSales;
SELECT p.ProductID, p.ProductName,
       COUNT(r.ReturnID) * 100.0 / COUNT(od.OrderDetailID) AS ReturnRate
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
LEFT JOIN Orders o ON od.OrderID = o.OrderID
LEFT JOIN Returns r ON o.OrderID = r.OrderID
GROUP BY p.ProductID
HAVING COUNT(r.ReturnID) * 100.0 / COUNT(od.OrderDetailID) > 10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductID	ProductName	ReturnRate
6	Product 6	66.66667
10	Product 10	87.50000
47	Product 47	100.00000

7.

```sql
-- 7. Identify cities that generated over ₹1,00,000 in revenue
USE FlipkartSales;
SELECT ShippingCity, SUM(od.SellingPrice * od.Quantity) AS Revenue
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY ShippingCity
HAVING SUM(od.SellingPrice * od.Quantity) > 100000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ShippingCity	Revenue

8.

```sql
-- 8. Find the average delivery time for each city
USE FlipkartSales;
SELECT ShippingCity,
       AVG(DATEDIFF(DeliveryDate, OrderDate)) AS AvgDeliveryTime
FROM Orders
WHERE DeliveryDate IS NOT NULL
GROUP BY ShippingCity;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ShippingCity	AvgDeliveryTime
East Josephville	46.0000
West Juan	-43.0000
Lake Matthew	-100.0000
Port Shannon	7.0000
Christinaton	37.0000
North Scott	7.0000
Lake Mark	6.0000
East Tonya	5.0000
Johnstonville	6.0000
Lake Tracy	8.0000
Mumbai	5.0000
Delhi	5.0000
Bangalore	4.0000
Hyderabad	5.0000
Chennai	6.0000
Kolkata	6.0000
Pune	8.0000
Jaipur	4.0000
Ahmedabad	5.0000
Surat	5.0000
Lucknow	4.0000
Nagpur	5.0000
Indore	5.0000

9.

```sql
-- 9. List all products that have been sold in more than 3 different cities
USE FlipkartSales;
SELECT od.ProductID, COUNT(DISTINCT o.ShippingCity) AS CityCount
FROM OrderDetails od
JOIN Orders o ON od.OrderID = o.OrderID
GROUP BY od.ProductID
HAVING COUNT(DISTINCT o.ShippingCity) > 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductID	CityCount
1	6
2	6
3	6
4	5
5	6
6	8
7	6
8	6
9	5
10	7
11	6
12	5
13	6
14	6
15	6

10.

```sql
-- 10. Show customers who purchased from more than 2 unique categories
USE FlipkartSales;
SELECT o.CustomerID, COUNT(DISTINCT p.CategoryID) AS CategoryCount
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY o.CustomerID
HAVING COUNT(DISTINCT p.CategoryID) > 2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	CategoryCount

➢ SECTION 2:

11.

```sql
-- 11. Join orders and products to show product names for each order
USE FlipkartSales;
SELECT o.OrderID, p.ProductName
FROM OrderDetails od
JOIN Orders o ON od.OrderID = o.OrderID
JOIN Products p ON od.ProductID = p.ProductID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

OrderID	ProductName
1	Product 27
2	Product 6
2	Product 10
3	Product 47
4	Product 34
5	Product 45
5	Product 40
6	Product 20
6	Product 39
7	Product 6
11	Product 4
12	Product 7
13	Product 3
14	Product 10
15	Product 6
16	Product 2
17	Product 8
18	Product 11
19	Product 5
20	Product 13
21	Product 12
22	Product 9
23	Product 1
24	Product 15
24	Product 15
25	Product 16
26	Product 3
27	Product 7
28	Product 14
29	Product 6
30	Product 2
31	Product 5
32	Product 4
33	Product 1
34	Product 10
35	Product 8
36	Product 9
37	Product 11

12.

```sql
-- 12. Display the customer name, product name, and quantity ordered
USE FlipkartSales;
SELECT c.CustomerName, p.ProductName, od.Quantity
FROM OrderDetails od
JOIN Orders o ON od.OrderID = o.OrderID
JOIN Products p ON od.ProductID = p.ProductID
JOIN Customers c ON o.CustomerID = c.CustomerID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerName	ProductName	Quantity
Rachel James	Product 27	4
Jessica Young	Product 6	9
Jessica Young	Product 10	1
Daniel Gallagher	Product 47	9
James Reed	Product 34	6
Lisa Hensley	Product 45	8
Lisa Hensley	Product 40	4
Karen Brooks	Product 20	1
Karen Brooks	Product 39	4
Franklin Garcia	Product 6	10
Franklin Garcia	Product 4	2
Henry Santiago	Product 7	1
Susan Lopez	Product 3	3
Catherine Bryant	Product 10	1
Ann Weaver	Product 6	5
Amy Griffin	Product 2	1
Teresa Willis	Product 8	2
James Reed	Product 11	3
Joan Lane	Product 5	1

13.

```sql
-- 13. Join customers and orders to find the top 5 cities by number of orders
USE FlipkartSales;
SELECT City, COUNT(*) AS OrderCount
FROM Orders o
JOIN Customers c ON o.CustomerID = c.CustomerID
GROUP BY City
ORDER BY OrderCount DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

City	OrderCount
Lake Crystal	3
South William	3
New Carolyn	2
South Christianport	2
Ramirezstad	2

14.

```
1   -- 14. Show all orders with customer email and shipping city
2   USE FlipkartSales;
3   SELECT o.OrderID, c.Email, o.ShippingCity
4   FROM Orders o
5   JOIN Customers c ON o.CustomerID = c.CustomerID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

OrderID	Email	ShippingCity
1	racheljames@example.com	East Josephville
2	josephhall@martin.com	West Juan
3	jamesmichael@munoz-roman.info	Lake Matthew
4	jimmyfox@benson.com	Port Shannon
5	williamsjeremy@galloway-wyatt.com	Christinaton
6	karenbrooks@example.com	North Scott
7	franklin.garcia@example.com	Lake Mark
8	amandasanchez@gray-mayo.net	East Tonya
9	jillrhodes@miller.com	Johnstonville
10	keithgibbs@hotmail.com	Lake Tracy
11	franklin.garcia@example.com	Mumbai
12	amandasanchez@gray-mayo.net	Delhi
13	martinrodriguez@bishop.com	Bangalore
14	catherinebryant@example.com	Hyderabad
15	annweaver@example.com	Chennai

15.

```
1   -- 15. List products with their supplier names and categories
2   USE FlipkartSales;
3   SELECT p.ProductName, s.SupplierName, c.CategoryName
4   FROM Products p
5   JOIN Suppliers s ON p.SupplierID = s.SupplierID
6   JOIN Categories c ON p.CategoryID = c.CategoryID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductName	SupplierName	CategoryName
Product 1	Ferrell, Jones and Lewis	Books
Product 2	Brewer, Bryant and Hickman	Home
Product 3	Kramer and Sons	Books
Product 4	Cooper Ltd	Electronics
Product 5	Morris and Sons	Home
Product 6	Allen-Foster	Books
Product 7	Stewart, Green and Santiago	Books
Product 8	Smith PLC	Books
Product 9	Stewart, Green and Santiago	Clothing
Product 10	Vazquez-Drake	Clothing

16.

```sql
-- 16. Join sales and products to show average price per category
USE FlipkartSales;
SELECT c.CategoryName, AVG(p.Price) AS AvgPrice
FROM Products p
JOIN Categories c ON p.CategoryID = c.CategoryID
GROUP BY c.CategoryName;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CategoryName	AvgPrice
Books	1086.120435
Home	2806.486500
Electronics	2699.638421
Clothing	1428.891000
Sports	1016.268333

17.

```sql
-- 17. Display all order details along with payment method from a third table
USE FlipkartSales;
SELECT od.*, o.PaymentMethod
FROM OrderDetails od
JOIN Orders o ON od.OrderID = o.OrderID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

OrderDetailID	OrderID	ProductID	Quantity	SellingPrice	PaymentMethod
1	1	27	4	4187.64	Cash
2	2	6	9	4579.08	Card
3	2	10	1	4811.03	Card
4	3	47	9	3458.12	UPI
5	4	34	6	1669.84	Cash
6	5	45	8	2449.39	UPI
7	5	40	4	2161.91	UPI
8	6	20	1	3322.53	Card
9	6	39	4	1348.34	Card
10	7	6	10	2885.62	Cash
11	11	4	2	1599.00	UPI
12	12	7	1	2299.00	Credit Card
13	13	3	3	750.00	Cash
14	14	10	1	8499.00	Net Banking
15	15	6	5	299.00	UPI
16	16	2	1	499.00	Credit Card
17	17	8	2	2199.00	Debit Card
18	18	11	3	1100.00	Net Banking
19	19	5	1	2799.00	Credit Card
20	20	13	2	1350.00	Cash

18.

```sql
-- 18. Find all customers and their total order amounts using a JOIN
USE FlipkartSales;
SELECT c.CustomerID, c.CustomerName, SUM(od.Quantity * od.SellingPrice) AS TotalAmount
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY c.CustomerID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	CustomerName	TotalAmount
20	Rachel James	19146.56
10	Jessica Young	46321.75
3	Daniel Gallagher	33119.08
9	James Reed	13319.04
4	Lisa Hensley	45240.76
13	Karen Brooks	9014.89
12	Franklin Garcia	32054.20
5	Henry Santiago	2299.00
8	Susan Lopez	2250.00
22	Catherine Bryant	8499.00
33	Ann Weaver	1495.00
18	Amy Griffin	499.00
45	Teresa Willis	4398.00
51	Joan Lane	2799.00
37	Denise Armstrong	2700.00
63	Anna Riley	449.00
26	Ashley Chapman	2796.00
74	Chad Torres	3998.00
14	Lawrence Foster	999.00
39	Martha Chapman	1797.00
71	Lillian Sherman	3750.00
50	Harry Riley	4598.00
77	Angela Watts	3799.00
60	Roy Garrett	897.00
31	Diana Larson	499.00
80	Todd Reynolds	5598.00
43	Jane Ramos	1599.00
36	Keith Spencer	5997.00
17	Nicholas Patterson	21993.00
55	Kathy Morrison	8796.00
65	Doris Peters	1398.00
24	Jeanette Reynolds	1100.00
90	Jared Bush	2700.00
7	Michael Green	3799.00
29	Marilyn Austin	2997.00
44	Andrew Hawkins	1500.00
88	Harold Pierce	2199.00
59	Sara Franklin	7996.00

19.

```sql
-- 19. Show product categories and corresponding supplier regions
USE FlipkartSales;
SELECT c.CategoryName, s.Region
FROM Products p
JOIN Categories c ON p.CategoryID = c.CategoryID
JOIN Suppliers s ON p.SupplierID = s.SupplierID
GROUP BY c.CategoryName, s.Region;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CategoryName	Region
Books	North
Home	South
Books	West
Electronics	West
Books	East
Clothing	East
Clothing	South
Electronics	East
Clothing	North
Sports	East
Electronics	South

20.

Limit to 1000 rows

```sql
-- 20. Join orders and returns to list orders that were returned
USE FlipkartSales;
SELECT o.OrderID, r.ReturnDate, r.Reason
FROM Orders o
JOIN Returns r ON o.OrderID = r.OrderID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

OrderID	ReturnDate	Reason
3	2024-01-27	Other
6	2024-06-22	Damaged
13	2024-02-10	Late Delivery
14	2024-03-18	Wrong Product
19	2024-05-30	Damaged
21	2024-06-20	Wrong Product
25	2024-04-05	Late Delivery
26	2024-04-15	Damaged
29	2024-02-23	Other
32	2024-06-10	Damaged
17	2023-06-30	Damaged
24	2023-07-07	Wrong Product
33	2023-07-15	Late delivery
38	2023-07-19	Other
40	2023-07-22	Late Delivery

➢ SECTION 3:

21.

```sql
-- 21. Categorize products into 'Low', 'Medium', and 'High' based on price
USE FlipkartSales;
SELECT ProductName, Price,
       CASE
         WHEN Price < 500 THEN 'Low'
         WHEN Price BETWEEN 500 AND 2000 THEN 'Medium'
         ELSE 'High'
       END AS PriceCategory
FROM Products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductName	Price	PriceCategory
Product 1	2321.71	High
Product 2	1913.89	Medium
Product 3	1381.84	Medium
Product 4	1492.67	Medium
Product 5	1559.13	Medium
Product 6	3368.44	High
Product 7	3981.35	High
Product 8	3075.79	High
Product 9	1555.85	Medium
Product 10	2163.11	High

22.

```sql
-- 22. Use CASE to label customers as 'New', 'Returning', or 'Loyal' based on order count
USE FlipkartSales;
SELECT CustomerID,
       COUNT(OrderID) AS TotalOrders,
       CASE
         WHEN COUNT(OrderID) = 1 THEN 'New'
         WHEN COUNT(OrderID) BETWEEN 2 AND 5 THEN 'Returning'
         ELSE 'Loyal'
       END AS CustomerType
FROM Orders
GROUP BY CustomerID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	TotalOrders	CustomerType
1	2	Returning
2	1	New
3	2	Returning
4	2	Returning
5	2	Returning
6	3	Returning
7	1	New
8	1	New
9	2	Returning
10	2	Returning

23.

```sql
-- 23. Create a flag for late deliveries (Delivered After Estimated Date)
USE FlipkartSales;
SELECT OrderID,
       EstimatedDeliveryDate,
       DeliveryDate,
       CASE
         WHEN DeliveryDate > EstimatedDeliveryDate THEN 'Late'
         ELSE 'On Time'
       END AS DeliveryStatus
FROM Orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

OrderID	EstimatedDeliveryDate	DeliveryDate	DeliveryStatus
1	2024-05-10	2024-05-08	On Time
2	2024-01-20	2024-01-19	On Time
3	2024-01-23	2024-01-25	Late
4	2024-01-28	2024-01-31	Late
5	2024-05-20	2024-05-22	Late
6	2024-06-15	2024-06-17	Late
7	2024-05-13	2024-05-14	Late
8	2024-03-05	2024-03-04	On Time
9	2024-01-18	2024-01-19	Late
10	2024-06-10	2024-06-12	Late
11	2023-06-20	2023-06-20	On Time
12	2023-06-21	2023-06-21	On Time
13	2023-06-22	2023-06-21	On Time
14	2023-06-23	2023-06-23	On Time
15	2023-06-24	2023-06-25	Late
16	2023-06-25	2023-06-26	Late
17	2023-06-26	2023-06-29	Late
18	2023-06-27	2023-06-26	On Time
19	2023-06-28	2023-06-28	On Time
20	2023-06-29	2023-06-29	On Time
21	2023-06-30	2023-06-29	On Time
22	2023-07-01	2023-07-01	On Time
23	2023-07-02	2023-07-02	On Time
24	2023-07-03	2023-07-06	Late
25	2023-07-04	2023-07-03	On Time
26	2023-07-05	2023-07-05	On Time
27	2023-07-06	2023-07-06	On Time
28	2023-07-07	2023-07-07	On Time
29	2023-07-08	2023-07-08	On Time
30	2023-07-09	2023-07-09	On Time
31	2023-07-10	2023-07-09	On Time
32	2023-07-11	2023-07-10	On Time
33	2023-07-12	2023-07-14	Late
34	2023-07-13	2023-07-12	On Time
35	2023-07-14	2023-07-14	On Time

24.

```sql
-- 24. Label orders as 'High Value' if amount > ₹10,000, else 'Normal'
USE FlipkartSales;
SELECT od.OrderID,
       SUM(od.Quantity * od.SellingPrice) AS OrderValue,
       CASE
         WHEN SUM(od.Quantity * od.SellingPrice) > 10000 THEN 'High Value'
         ELSE 'Normal'
       END AS ValueLabel
FROM OrderDetails od
GROUP BY od.OrderID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

OrderID	OrderValue	ValueLabel
1	16750.56	High Value
2	46022.75	High Value
3	31123.08	High Value
4	10019.04	High Value
5	28242.76	High Value
6	8715.89	Normal
7	28856.20	High Value
11	3198.00	Normal
12	2299.00	Normal
13	2250.00	Normal
14	8499.00	Normal
15	1495.00	Normal

25.

```sql
-- 25. Use a CASE to segment revenue ranges for cities
USE FlipkartSales;
SELECT ShippingCity,
       SUM(od.Quantity * od.SellingPrice) AS Revenue,
       CASE
         WHEN SUM(od.Quantity * od.SellingPrice) < 50000 THEN 'Low'
         WHEN SUM(od.Quantity * od.SellingPrice) BETWEEN 50000 AND 100000 THEN 'Medium'
         ELSE 'High'
       END AS RevenueSegment
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY ShippingCity;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ShippingCity	Revenue	RevenueSegment
East Josephville	16750.56	Low
West Juan	46022.75	Low
Lake Matthew	31123.08	Low
Port Shannon	10019.04	Low
Christinaton	28242.76	Low
North Scott	8715.89	Low
Lake Mark	28856.20	Low

➢ SECTION 4:

26.

```sql
-- 26. Use ROW_NUMBER() to rank products by sales within each category
USE FlipkartSales;
SELECT *,
       ROW_NUMBER() OVER (PARTITION BY CategoryID ORDER BY TotalSales DESC) AS RankInCategory
FROM (
  SELECT p.ProductID, p.CategoryID, SUM(od.Quantity) AS TotalSales
  FROM OrderDetails od
  JOIN Products p ON od.ProductID = p.ProductID
  GROUP BY p.ProductID, p.CategoryID
) AS ProductSales;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductID	CategoryID	TotalSales	RankInCategory
11	1	10	1
16	1	9	2
4	1	6	3
10	2	13	1
9	2	12	2
47	2	9	3
12	2	9	4
27	2	4	5
2	3	13	1
13	3	10	2
5	3	9	3

27.

```sql
USE FlipkartSales;
SELECT * FROM (
  SELECT ShippingCity,
         SUM(od.Quantity * od.SellingPrice) AS Revenue,
         RANK() OVER (ORDER BY SUM(od.Quantity * od.SellingPrice) DESC)
  FROM Orders o
  JOIN OrderDetails od ON o.OrderID = od.OrderID
  GROUP BY ShippingCity
) AS RankedCities
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ShippingCity	Revenue	RANK() OVER (ORDER BY SUM(od.Quantity * od.SellingPrice) DESC)
West Juan	46022.75	1
Lake Matthew	31123.08	2
Lake Mark	28856.20	3
Christinaton	28242.76	4
Udaipur	25497.00	5
Nellore	16998.00	6
Gandhinagar	16998.00	6
Jabalpur	16998.00	6
Barasat	16998.00	6

28.

```sql
-- 28. Calculate cumulative revenue day-wise using SUM() OVER()
USE FlipkartSales;
SELECT OrderDate,
       SUM(od.Quantity * od.SellingPrice) AS DailyRevenue,
       SUM(SUM(od.Quantity * od.SellingPrice)) OVER (ORDER BY OrderDate) AS CumulativeRevenue
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY OrderDate;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

OrderDate	DailyRevenue	CumulativeRevenue
2023-06-15	3198.00	3198.00
2023-06-16	2299.00	5497.00
2023-06-17	2250.00	7747.00
2023-06-18	8499.00	16246.00
2023-06-19	1495.00	17741.00
2023-06-20	499.00	18240.00
2023-06-21	4398.00	22638.00
2023-06-22	3300.00	25938.00
2023-06-23	2799.00	28737.00
2023-06-24	2700.00	31437.00
2023-06-25	449.00	31886.00

29.

```sql
-- 29. Show each customer's average order value using AVG() OVER()
USE FlipkartSales;
SELECT
    CustomerID,
    OrderID,
    OrderValue,
    AVG(OrderValue) OVER (PARTITION BY CustomerID) AS AvgOrderValue
FROM (
    SELECT
        o.CustomerID,
        o.OrderID,
        SUM(od.Quantity * od.SellingPrice) AS OrderValue
    FROM Orders o
    JOIN OrderDetails od ON o.OrderID = od.OrderID
    GROUP BY o.CustomerID, o.OrderID
) AS OrderTotals;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	OrderID	OrderValue	AvgOrderValue
1	80	7598.00	7598.000000
2	48	2700.00	2700.000000
3	3	31123.08	16559.540000
3	70	1996.00	16559.540000

30.

```
-- 30. Partition customer order history and assign order sequence using ROW_NUMBER()
USE FlipkartSales;
SELECT CustomerID, OrderID, OrderDate,
       ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY OrderDate) AS OrderSequence
FROM Orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	OrderID	OrderDate	OrderSequence
1	80	2023-08-23	1
1	9	2024-01-13	2
2	48	2023-07-22	1
3	70	2023-08-13	1
3	3	2024-05-04	2
4	77	2023-08-20	1
4	5	2024-04-15	2
5	12	2023-06-16	1
5	8	2024-02-28	2
6	66	2023-08-09	1
6	93	2023-09-05	2
6	10	2024-06-04	3
7	39	2023-07-13	1
8	13	2023-06-17	1
9	18	2023-06-22	1
9	4	2024-01-24	2
10	58	2023-08-01	1
10	2	2024-03-02	2
11	81	2023-08-24	1
12	11	2023-06-15	1
12	7	2024-05-08	2
13	42	2023-07-16	1
13	6	2024-06-10	2
14	24	2023-06-28	1
15	69	2023-08-12	1
16	82	2023-08-25	1
17	34	2023-07-08	1
17	50	2023-07-24	2
18	16	2023-06-20	1
19	62	2023-08-05	1
20	56	2023-07-30	1
20	1	2024-03-23	2
21	90	2023-09-02	1
22	14	2023-06-18	1
23	45	2023-07-19	1
24	37	2023-07-11	1
25	78	2023-08-21	1
26	22	2023-06-26	1
27	51	2023-07-25	1
28	84	2023-08-27	1
29	40	2023-07-14	1
30	75	2023-08-18	1
31	30	2023-07-04	1
32	64	2023-08-07	1
33	15	2023-06-19	1

31.

```sql
-- 31. Use DENSE_RANK() to find second-best performing product in each region
USE FlipkartSales;
SELECT * FROM (
  SELECT s.Region, p.ProductID,
         SUM(od.Quantity * od.SellingPrice) AS Revenue,
         DENSE_RANK() OVER (PARTITION BY s.Region ORDER BY SUM(od.Quantity * od.SellingPrice) DESC)
  FROM OrderDetails od
  JOIN Products p ON od.ProductID = p.ProductID
  JOIN Suppliers s ON p.SupplierID = s.SupplierID
  GROUP BY s.Region, p.ProductID
) AS RankedProducts;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Region	ProductID	Revenue	DENSE_RANK() OVER (PARTITION BY s.Region ORDER BY SUM(od.Quantity * od.SellingPrice) DESC)
East	6	74552.92	1
East	7	18392.00	2
East	15	15984.00	3
East	11	11000.00	4
East	9	8388.00	5
East	39	5393.36	6
North	8	32985.00	1
North	1	25987.00	2
North	45	19595.12	3
North	34	10019.04	4
North	12	4041.00	5
South	10	106799.03	1
South	5	25191.00	2
South	27	16750.56	3
South	13	13500.00	4
South	40	8647.64	5
South	2	6487.00	6

32.

```sql
-- 32. Use NTILE(4) to divide products into 4 price bands
USE FlipkartSales;
SELECT ProductID, ProductName, Price,
       NTILE(4) OVER (ORDER BY Price) AS PriceBand
FROM Products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductID	ProductName	Price	PriceBand
94	Product 94	455.00	1
34	Product 34	475.00	1
54	Product 54	499.99	1
14	Product 14	499.99	1
79	Product 79	510.00	1
39	Product 39	525.50	1
24	Product 24	545.25	1
69	Product 69	570.00	1
59	Product 59	580.45	1
99	Product 99	599.00	1

33.

```sql
-- 33. Compute running total of quantity sold per category
USE FlipkartSales;
SELECT c.CategoryID, c.CategoryName, o.OrderDate,
       SUM(od.Quantity) AS DailyQty,
       SUM(SUM(od.Quantity)) OVER (PARTITION BY c.CategoryID ORDER BY o.OrderDate) AS RunningTotalQty
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
JOIN Categories c ON p.CategoryID = c.CategoryID
JOIN Orders o ON o.OrderID = od.OrderID
GROUP BY c.CategoryID, c.CategoryName, o.OrderDate;
```

Result Grid

CategoryID	CategoryName	OrderDate	DailyQty	RunningTotalQty
1	Electronics	2023-06-15	2	2
1	Electronics	2023-06-22	3	5
1	Electronics	2023-06-29	3	8
1	Electronics	2023-07-06	1	9
1	Electronics	2023-07-11	1	10
1	Electronics	2023-07-28	1	11
1	Electronics	2023-07-29	3	14
1	Electronics	2023-07-30	4	18
1	Electronics	2023-08-12	1	19
1	Electronics	2023-08-14	1	20
1	Electronics	2023-08-25	1	21
1	Electronics	2023-08-29	2	23
1	Electronics	2023-09-01	1	24
1	Electronics	2023-09-09	1	25
2	Clothing	2023-06-18	1	1
2	Clothing	2023-06-25	1	2
2	Clothing	2023-06-26	4	6
2	Clothing	2023-07-08	2	8
2	Clothing	2023-07-10	2	10
2	Clothing	2023-07-21	3	13

34.

```sql
-- 34. Show daily revenue and previous day's revenue using LAG()
USE FlipkartSales;
SELECT OrderDate,
       SUM(od.Quantity * od.SellingPrice) AS Revenue,
       LAG(SUM(od.Quantity * od.SellingPrice)) OVER (ORDER BY OrderDate) AS PreviousDayRevenue
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY OrderDate;
```

Result Grid

OrderDate	Revenue	PreviousDayRevenue
2023-06-15	3198.00	NULL
2023-06-16	2299.00	3198.00
2023-06-17	2250.00	2299.00
2023-06-18	8499.00	2250.00
2023-06-19	1495.00	8499.00
2023-06-20	499.00	1495.00
2023-06-21	4398.00	499.00
2023-06-22	3300.00	4398.00
2023-06-23	2799.00	3300.00
2023-06-24	2700.00	2799.00
2023-06-25	449.00	2700.00

35.

```sql
-- 35. Find the revenue change from previous day using LAG() and LEAD()
USE FlipkartSales;
SELECT OrderDate,
       SUM(od.Quantity * od.SellingPrice) AS Revenue,
       LAG(SUM(od.Quantity * od.SellingPrice)) OVER (ORDER BY OrderDate) AS PrevDay,
       LEAD(SUM(od.Quantity * od.SellingPrice)) OVER (ORDER BY OrderDate) AS NextDay,
       SUM(od.Quantity * od.SellingPrice) - LAG(SUM(od.Quantity * od.SellingPrice)) OVER (ORDER BY OrderDate) AS RevenueChange
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY OrderDate;
```

OrderDate	Revenue	PrevDay	NextDay	RevenueChange
2023-06-15	3198.00	NULL	2299.00	NULL
2023-06-16	2299.00	3198.00	2250.00	-899.00
2023-06-17	2250.00	2299.00	8499.00	-49.00
2023-06-18	8499.00	2250.00	1495.00	6249.00
2023-06-19	1495.00	8499.00	499.00	-7004.00
2023-06-20	499.00	1495.00	4398.00	-996.00
2023-06-21	4398.00	499.00	3300.00	3899.00
2023-06-22	3300.00	4398.00	2799.00	-1098.00
2023-06-23	2799.00	3300.00	2700.00	-501.00
2023-06-24	2700.00	2799.00	449.00	-99.00
2023-06-25	449.00	2700.00	2796.00	-2251.00
2023-06-26	2796.00	449.00	3998.00	2347.00
2023-06-27	3998.00	2796.00	999.00	1202.00
2023-06-28	999.00	3998.00	1797.00	-2999.00
2023-06-29	1797.00	999.00	3750.00	798.00
2023-06-30	3750.00	1797.00	4598.00	1953.00
2023-07-01	4598.00	3750.00	3799.00	848.00
2023-07-02	3799.00	4598.00	897.00	-799.00
2023-07-03	897.00	3799.00	499.00	-2902.00
2023-07-04	499.00	897.00	5598.00	-398.00
2023-07-05	5598.00	499.00	1599.00	5099.00
2023-07-06	1599.00	5598.00	5997.00	-3999.00
2023-07-07	5997.00	1599.00	16998.00	4398.00
2023-07-08	16998.00	5997.00	8796.00	11001.00
2023-07-09	8796.00	16998.00	1398.00	-8202.00
2023-07-10	1398.00	8796.00	1100.00	-7398.00
2023-07-11	1100.00	1398.00	2700.00	-298.00
2023-07-12	2700.00	1100.00	3799.00	1600.00
2023-07-13	3799.00	2700.00	2997.00	1099.00
2023-07-14	2997.00	3799.00	1500.00	-802.00
2023-07-15	1500.00	2997.00	299.00	-1497.00
2023-07-16	299.00	1500.00	2199.00	-1201.00

➢ SECTION 5:

36.

```sql
-- 36. Write a CTE to find customers who placed more than 3 orders
USE FlipkartSales;
WITH CustomerOrders AS (
  SELECT CustomerID, COUNT(*) AS OrderCount
  FROM Orders
  GROUP BY CustomerID
)
SELECT * FROM CustomerOrders WHERE OrderCount > 3;
```

CustomerID	OrderCount

37.

```sql
-- 37. Use CTE to identify the top product by quantity in each category
USE FlipkartSales;
WITH ProductSales AS (
  SELECT
    p.ProductID,
    p.CategoryID,
    SUM(od.Quantity) AS TotalQuantity
  FROM Products p
  JOIN OrderDetails od ON p.ProductID = od.ProductID
  GROUP BY p.ProductID, p.CategoryID
),
RankedProducts AS (
  SELECT
    ProductID,
    CategoryID,
    TotalQuantity,
    RANK() OVER (PARTITION BY CategoryID ORDER BY TotalQuantity DESC) AS rnk
  FROM ProductSales
)
SELECT *
FROM RankedProducts
WHERE rnk = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductID	CategoryID	TotalQuantity	rnk
11	1	10	1
10	2	13	1
2	3	13	1
6	4	34	1
15	5	16	1

38.

```sql
-- 38. Create a CTE to filter all sales in the last 30 days, and calculate revenue
USE FlipkartSales;
WITH RecentSales AS (
  SELECT * FROM Orders
  WHERE OrderDate >= CURDATE() - INTERVAL 30 DAY
)
SELECT SUM(od.Quantity * od.SellingPrice) AS Revenue
FROM RecentSales rs
JOIN OrderDetails od ON rs.OrderID = od.OrderID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Revenue
NULL

39.

```sql
USE FlipkartSales;
WITH MonthlySales AS (
  SELECT c.CategoryID, MONTH(o.OrderDate) AS Month, SUM(od.Quantity) AS TotalQty
  FROM OrderDetails od
  JOIN Orders o ON od.OrderID = o.OrderID
  JOIN Products p ON od.ProductID = p.ProductID
  JOIN Categories c ON p.CategoryID = c.CategoryID
  GROUP BY c.CategoryID, MONTH(o.OrderDate)
), Diff AS (
  SELECT *, LAG(TotalQty) OVER (PARTITION BY CategoryID ORDER BY Month) AS PrevMonthQty
  FROM MonthlySales
)
SELECT * FROM Diff WHERE TotalQty < PrevMonthQty;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CategoryID	Month	TotalQty	PrevMonthQty
1	8	5	10
1	9	2	5
2	6	6	9
2	9	4	12
3	9	6	11
4	9	11	28
5	6	2	12
5	8	6	8
5	9	1	6

40.

```sql
-- 40. Write a CTE that calculates average order value per customer
USE FlipkartSales;
WITH CustomerOrderValue AS (
  SELECT c.CustomerID, SUM(od.Quantity * od.SellingPrice) AS TotalValue, COUNT(DISTINCT o.OrderID) AS TotalOrders
  FROM Customers c
  JOIN Orders o ON c.CustomerID = o.CustomerID
  JOIN OrderDetails od ON o.OrderID = od.OrderID
  GROUP BY c.CustomerID
)
SELECT CustomerID, TotalValue / TotalOrders AS AvgOrderValue
FROM CustomerOrderValue;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	AvgOrderValue
1	7598.000000
2	2700.000000
3	16559.540000
4	22620.380000
5	2299.000000
6	9174.000000
7	3799.000000
8	2250.000000
9	6659.520000
10	23160.875000
11	2997.000000
12	16027.100000
13	4507.445000
14	999.000000
15	1100.000000

41.

```sql
-- 41. Use a CTE to get top 5 cities by number of returns
USE FlipkartSales;
WITH ReturnCounts AS (
  SELECT o.ShippingCity, COUNT(r.ReturnID) AS TotalReturns
  FROM Returns r
  JOIN Orders o ON r.OrderID = o.OrderID
  GROUP BY o.ShippingCity
)
SELECT * FROM ReturnCounts
ORDER BY TotalReturns DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ShippingCity	TotalReturns
Amritsar	2
Nashik	2
Patna	2
Ahmedabad	2
Bangalore	2

42.

```sql
-- 42. Combine multiple CTEs to compare returned vs delivered orders
USE FlipkartSales;
WITH ReturnedOrders AS (
  SELECT OrderID FROM Returns
), DeliveredOrders AS (
  SELECT OrderID FROM Orders WHERE Status = 'Delivered'
)
SELECT 'Returned' AS StatusType, COUNT(*) AS Count FROM ReturnedOrders
UNION ALL
SELECT 'Delivered' AS StatusType, COUNT(*) AS Count FROM DeliveredOrders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

StatusType	Count
Returned	100
Delivered	60

43.

```sql
-- 43. Use a recursive CTE to simulate product hierarchy (if applicable)
USE FlipkartSales;
SELECT o.CustomerID, COUNT(DISTINCT p.CategoryID) AS CategoryCount
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY o.CustomerID
HAVING COUNT(DISTINCT p.CategoryID) > 2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	CategoryCount

44.

```sql
-- 44. Use a CTE to calculate average delivery delay per city
USE FlipkartSales;
WITH DeliveryDelays AS (
  SELECT ShippingCity, DATEDIFF(DeliveryDate, EstimatedDeliveryDate) AS Delay
  FROM Orders
  WHERE DeliveryDate IS NOT NULL
)
SELECT ShippingCity, AVG(Delay) AS AvgDelay FROM DeliveryDelays GROUP BY ShippingCity;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ShippingCity	AvgDelay
East Josephville	-2.0000
West Juan	-1.0000
Lake Matthew	2.0000
Port Shannon	3.0000
Christinaton	2.0000
North Scott	2.0000
Lake Mark	1.0000
East Tonya	-1.0000
Johnstonville	1.0000
Lake Tracy	2.0000
Mumbai	0.0000
Delhi	0.0000
Bangalore	-1.0000
Hyderabad	0.0000
Chennai	1.0000
Kolkata	1.0000
Pune	3.0000
Jaipur	-1.0000

45.

```sql
-- 45. Create a CTE to find customers who've bought from at least 3 categories
USE FlipkartSales;
WITH CustomerCategories AS (
  SELECT o.CustomerID, COUNT(DISTINCT p.CategoryID) AS CategoryCount
  FROM Orders o
  JOIN OrderDetails od ON o.OrderID = od.OrderID
  JOIN Products p ON od.ProductID = p.ProductID
  GROUP BY o.CustomerID
)
SELECT * FROM CustomerCategories WHERE CategoryCount >= 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	CategoryCount

➢ SECTION 6:

46.

```sql
-- 46. Create a view to show monthly category-wise revenue
USE FlipkartSales;
CREATE VIEW MonthlyCategoryRevenue AS
SELECT DATE_FORMAT(o.OrderDate, '%Y-%m') AS OrderMonth,
       c.CategoryName,
       SUM(od.SellingPrice * od.Quantity) AS Revenue
FROM OrderDetails od
JOIN Orders o ON od.OrderID = o.OrderID
JOIN Products p ON od.ProductID = p.ProductID
JOIN Categories c ON p.CategoryID = c.CategoryID
GROUP BY OrderMonth, c.CategoryName;
```

```
58 15:35:04 CREATE VIEW MonthlyCategoryRevenue AS SELECT DATE_FORMAT(o.OrderDate, '%Y-%m') AS OrderMonth,  c.CategoryNa...  0 row(s) affected
```

47.

```sql
-- 47. Create a view for customer-wise total and average spending
USE FlipkartSales;
CREATE VIEW CustomerSpending AS
SELECT c.CustomerID, c.CustomerName,
       SUM(od.SellingPrice * od.Quantity) AS TotalSpending,
       AVG(od.SellingPrice * od.Quantity) AS AvgSpending
FROM Customers c
JOIN Orders o ON c.CustomerID = o.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY c.CustomerID;
```

```
60  15:37:49  CREATE VIEW CustomerSpending AS SELECT c.CustomerID, c.CustomerName,  SUM(od.SellingPrice * od.Quantity) AS TotalS...  0 row(s) affected
```

48.

```sql
-- 48. Create a view to analyze top 10 products by quantity
USE FlipkartSales;
CREATE VIEW TopProductsByQuantity AS
SELECT p.ProductID, p.ProductName,
       SUM(od.Quantity) AS TotalQuantity
FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID
GROUP BY p.ProductID
ORDER BY TotalQuantity DESC
LIMIT 10;
```

```
62  15:40:11  CREATE VIEW TopProductsByQuantity AS SELECT p.ProductID, p.ProductName,  SUM(od.Quantity) AS TotalQuantity FROM ...  0 row(s) affected
```

49.

```sql
-- 49. Define a view that lists all late deliveries
USE FlipkartSales;
CREATE VIEW LateDeliveries AS
SELECT * FROM Orders
WHERE DeliveryDate > EstimatedDeliveryDate;
```

```
64  15:41:30  CREATE VIEW LateDeliveries AS SELECT * FROM Orders WHERE DeliveryDate > EstimatedDeliveryDate  0 row(s) affected
```

50.

```sql
-- 50. Build a view showing cumulative monthly revenue across all regions
USE FlipkartSales;
CREATE VIEW MonthlyRevenueByRegion AS
SELECT DATE_FORMAT(o.OrderDate, '%Y-%m') AS OrderMonth,
       c.Region,
       SUM(od.SellingPrice * od.Quantity) AS Revenue
FROM Orders o
JOIN Customers c ON o.CustomerID = c.CustomerID
JOIN OrderDetails od ON o.OrderID = od.OrderID
GROUP BY OrderMonth, c.Region;
```

```
66 15:43:37 CREATE VIEW MonthlyRevenueByRegion AS SELECT DATE_FORMAT(o.OrderDate, '%Y-%m') AS OrderM... 0 row(s) affected
```

➢ SECTION 7:

51.

```sql
-- 51. Find products whose sales are higher than the average product sales
USE FlipkartSales;
SELECT ProductID, SUM(Quantity) AS TotalSales
FROM OrderDetails
GROUP BY ProductID
HAVING SUM(Quantity) > (SELECT AVG(Sales) FROM (SELECT SUM(Quantity) AS Sales FROM OrderDetails GROUP BY ProductID) AS Sub);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductID	TotalSales
1	13
2	13
3	22
6	34
8	15
9	12
10	13
15	16

52.

```
1    -- 52. List customers who never returned any orders
2    USE FlipkartSales;
3    SELECT DISTINCT c.CustomerID, c.CustomerName
4    FROM Customers c
5    JOIN Orders o ON c.CustomerID = o.CustomerID
6    WHERE o.OrderID NOT IN (SELECT OrderID FROM Returns);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conter

CustomerID	CustomerName
1	Allison Hill
4	Lisa Hensley
5	Henry Santiago
6	Lisa Johnson
9	James Reed
10	Jessica Young
12	Franklin Garcia
20	Rachel James
33	Ann Weaver

53.

```
1    -- 53. Find orders that have the highest order value per customer
2    USE FlipkartSales;
3    SELECT o.OrderID, o.CustomerID
4    FROM Orders o
5    JOIN OrderDetails od ON o.OrderID = od.OrderID
6    GROUP BY o.OrderID, o.CustomerID
7    HAVING SUM(od.Quantity * od.SellingPrice) = (
8      SELECT MAX(Total) FROM (
9        SELECT o2.CustomerID, SUM(od2.Quantity * od2.SellingPrice) AS Total
10       FROM Orders o2
11       JOIN OrderDetails od2 ON o2.OrderID = od2.OrderID
12       GROUP BY o2.OrderID
13     ) AS OrderTotals WHERE OrderTotals.CustomerID = o.CustomerID
14   );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

OrderID	CustomerID
1	20
2	10
3	3
4	9
5	4
6	13
7	12
12	5
13	8
14	22
15	33
16	18
17	45

54.

```sql
-- 54. Get cities that contributed to more than 10% of total revenue
USE FlipkartSales;
WITH CityRevenue AS (
  SELECT ShippingCity, SUM(od.SellingPrice * od.Quantity) AS Revenue
  FROM Orders o
  JOIN OrderDetails od ON o.OrderID = od.OrderID
  GROUP BY ShippingCity
),
TotalRevenue AS (
  SELECT SUM(Revenue) AS Total FROM CityRevenue
)
SELECT cr.ShippingCity, cr.Revenue
FROM CityRevenue cr, TotalRevenue tr
WHERE cr.Revenue > tr.Total * 0.10;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ShippingCity	Revenue

55.

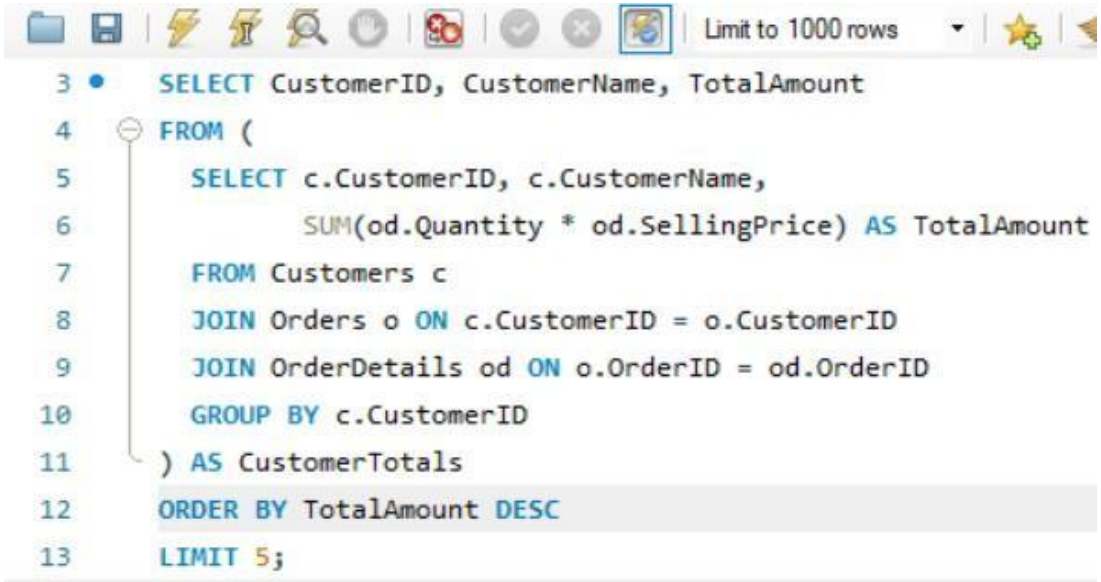
```sql
-- 55. Identify products that are not in any orders (zero sales)
USE FlipkartSales;
SELECT * FROM Products
WHERE ProductID NOT IN (SELECT DISTINCT ProductID FROM OrderDetails);
```

Result Grid | Filter Rows: | Edit: | Export/Import:

ProductID	ProductName	CategoryID	SupplierID	Price
17	Product 17	2	17	1599.00
18	Product 18	3	18	2799.00
19	Product 19	4	19	699.99
21	Product 21	1	21	3299.00
22	Product 22	2	22	1249.50
23	Product 23	3	23	3499.99
24	Product 24	4	24	545.25
25	Product 25	5	25	1045.75
26	Product 26	1	26	1799.00
28	Product 28	3	28	2675.00
29	Product 29	4	29	799.99
30	Product 30	5	30	1100.00
31	Product 31	1	31	3999.99
32	Product 32	2	32	1499.50
33	Product 33	3	33	2200.00
35	Product 35	5	35	950.00
36	Product 36	1	36	2850.00
37	Product 37	2	37	1120.99
38	Product 38	3	38	3150.75
41	Product 41	1	41	2899.99
42	Product 42	2	42	1345.50
43	Product 43	3	43	3100.00
44	Product 44	4	44	675.75
46	Product 46	1	46	2045.00

48	Product 48	3	48	2950.49
49	Product 49	4	49	899.00
50	Product 50	5	50	1150.00
51	Product 51	1	51	3199.00
52	Product 52	2	52	1490.90
53	Product 53	3	53	3300.00
54	Product 54	4	54	499.99
55	Product 55	5	55	1075.00
56	Product 56	1	56	2500.00
57	Product 57	2	57	1200.00
58	Product 58	3	58	2699.99
59	Product 59	4	59	580.45
60	Product 60	5	60	1000.00
61	Product 61	1	61	2999.99
62	Product 62	2	62	1390.00
63	Product 63	3	63	3100.00
64	Product 64	4	64	620.99
65	Product 65	5	65	980.00
66	Product 66	1	66	2345.00
67	Product 67	2	67	1299.99
68	Product 68	3	68	2850.25
69	Product 69	4	69	570.00
70	Product 70	5	70	1099.90
71	Product 71	1	71	3250.00

56.



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	CustomerName	TotalAmount
10	Jessica Young	46321.75
4	Lisa Hensley	45240.76
3	Daniel Gallagher	33119.08
12	Franklin Garcia	32054.20
91	Tina Craig	25497.00

57.

```sql
-- 57. Show all categories that have more than 5 products with zero returns
USE FlipkartSales;
SELECT c.CategoryName
FROM Products p
JOIN Categories c ON p.CategoryID = c.CategoryID
WHERE p.ProductID NOT IN (
  SELECT DISTINCT od.ProductID
  FROM OrderDetails od
  JOIN Orders o ON od.OrderID = o.OrderID
  JOIN Returns r ON o.OrderID = r.OrderID
)
GROUP BY c.CategoryID
HAVING COUNT(p.ProductID) > 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CategoryName
Electronics
Clothing
Home
Books
Sports

58.

```sql
-- 58. Find the latest order placed by each customer
USE FlipkartSales;
SELECT o.CustomerID, MAX(o.OrderDate) AS LatestOrder
FROM Orders o
GROUP BY o.CustomerID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conte

CustomerID	LatestOrder
1	2024-01-13
2	2023-07-22
3	2024-05-04
4	2024-04-15
5	2024-02-28
6	2024-06-04
7	2023-07-13
8	2023-06-17
9	2024-01-24
10	2024-03-02
11	2023-08-24
12	2024-05-08
13	2024-06-10
14	2023-06-28
15	2023-08-12
16	2023-08-25
17	2023-07-24
18	2023-06-20
19	2023-08-05
20	2024-03-23
21	2023-09-02
22	2023-06-18
23	2023-07-19
24	2023-07-11

25	2023-08-21
26	2023-06-26
27	2023-07-25
28	2023-08-27
29	2023-07-14
30	2023-08-18
31	2023-07-04
32	2023-08-07
33	2023-06-19
34	2023-08-03
35	2023-08-15
36	2023-07-07
37	2023-06-24
38	2023-07-27
39	2023-06-29
41	2023-07-31
42	2023-08-29
43	2023-07-06
44	2023-07-15
45	2023-06-21
46	2023-08-02
47	2023-09-01
49	2023-09-07
50	2023-07-01
51	2023-06-23
53	2023-09-04
54	2023-08-16
55	2023-07-09
57	2023-09-10
58	2023-07-23
59	2023-07-18
60	2023-07-03
62	2023-08-08
63	2023-06-25
64	2023-08-30
65	2023-07-10
66	2023-07-29
67	2023-08-19
68	2023-09-11
69	2023-07-20
71	2023-06-30
73	2023-08-28
74	2023-06-27
75	2023-09-03
76	2023-07-28
77	2023-07-02
78	2023-08-11
79	2023-09-12
80	2023-07-05
81	2023-08-14
84	2023-08-31
85	2023-07-26
86	2023-08-26
87	2023-09-06
88	2023-07-17
89	2023-08-22
90	2023-07-12
91	2023-07-21
92	2023-08-04
93	2023-09-09
94	2023-09-08
95	2023-08-10
99	2023-08-06
100	2023-08-17

59.

```
1   -- 59. List products that were sold at a higher-than-average price
2   USE FlipkartSales;
3   SELECT p.ProductID, p.ProductName
4   FROM Products p
5   JOIN OrderDetails od ON p.ProductID = od.ProductID
6   GROUP BY p.ProductID
7   HAVING AVG(od.SellingPrice) > (
8     SELECT AVG(SellingPrice) FROM OrderDetails
9   );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductID	ProductName
27	Product 27
10	Product 10
47	Product 47
45	Product 45
40	Product 40
20	Product 20
7	Product 7
8	Product 8
5	Product 5
14	Product 14

60.

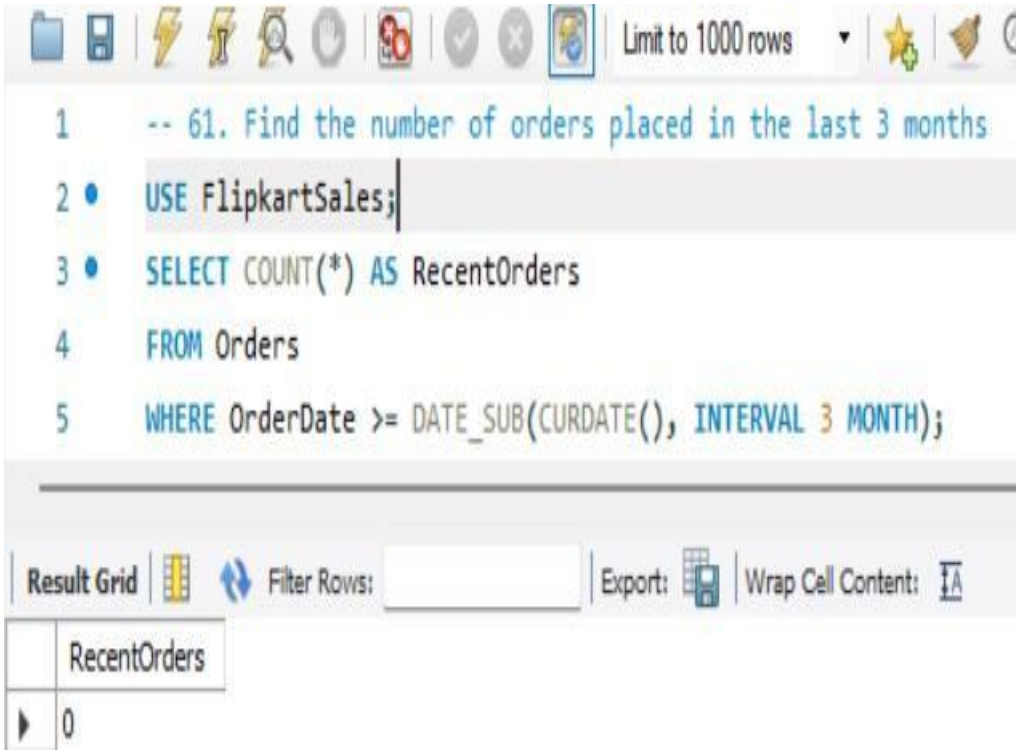
```
1   -- 60. Display customers whose average order value is above ₹2,000
2   USE FlipkartSales;
3   SELECT c.CustomerID, c.CustomerName,
4          AVG(od.SellingPrice * od.Quantity) AS AvgOrderValue
5   FROM Customers c
6   JOIN Orders o ON c.CustomerID = o.CustomerID
7   JOIN OrderDetails od ON o.OrderID = od.OrderID
8   GROUP BY c.CustomerID
9   HAVING AVG(od.SellingPrice * od.Quantity) > 2000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	CustomerName	AvgOrderValue
20	Rachel James	9573.280000
10	Jessica Young	15440.583333
3	Daniel Gallagher	16559.540000
9	James Reed	6659.520000
4	Lisa Hensley	15080.253333
13	Karen Brooks	3004.963333
12	Franklin Garcia	16027.100000
5	Henry Santiago	2299.000000
8	Susan Lopez	2250.000000
22	Catherine Bryant	8499.000000
45	Teresa Willis	4398.000000
51	Joan Lane	2799.000000
37	Denise Armstrong	2700.000000
26	Ashley Chapman	2796.000000
74	Chad Torres	3998.000000
71	Lillian Sherman	3750.000000
50	Harry Riley	4598.000000
77	Angela Watts	3799.000000
80	Todd Reynolds	5598.000000

➢ SECTION 8:

61.

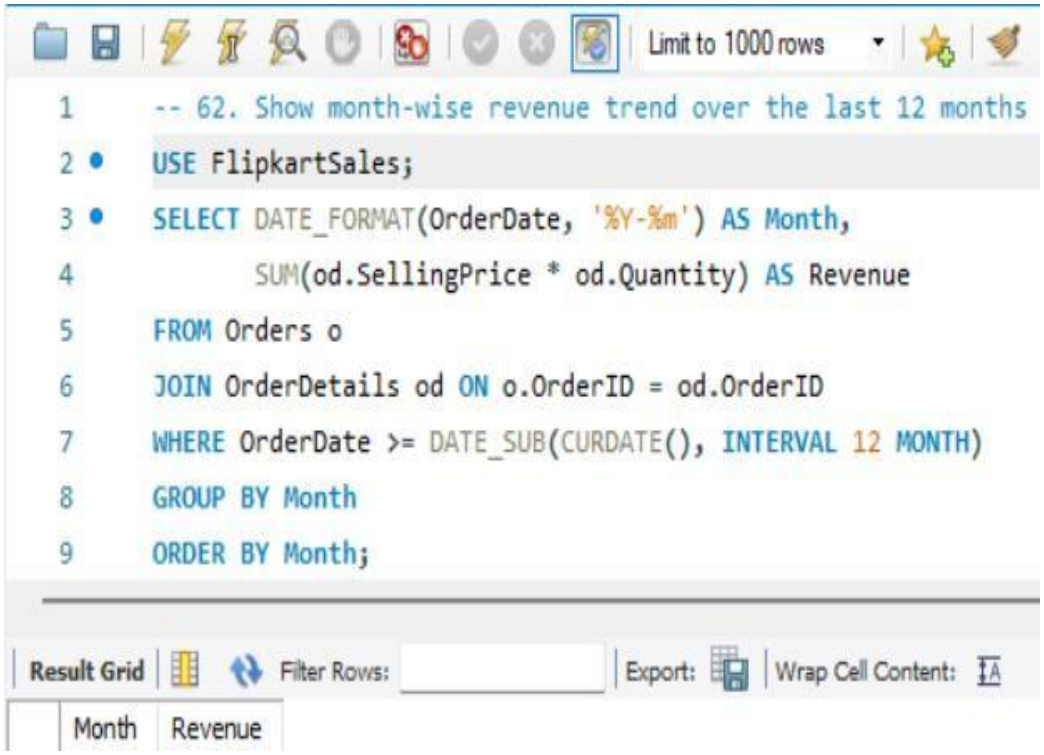


```
1   -- 61. Find the number of orders placed in the last 3 months
2   USE FlipkartSales;
3   SELECT COUNT(*) AS RecentOrders
4   FROM Orders
5   WHERE OrderDate >= DATE_SUB(CURDATE(), INTERVAL 3 MONTH);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

RecentOrders
0

62.



```
1   -- 62. Show month-wise revenue trend over the last 12 months
2   USE FlipkartSales;
3   SELECT DATE_FORMAT(OrderDate, '%Y-%m') AS Month,
4          SUM(od.SellingPrice * od.Quantity) AS Revenue
5   FROM Orders o
6   JOIN OrderDetails od ON o.OrderID = od.OrderID
7   WHERE OrderDate >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)
8   GROUP BY Month
9   ORDER BY Month;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Month	Revenue

63.

```sql
-- 63. Calculate average delivery time per product
USE FlipkartSales;
SELECT p.ProductID, p.ProductName,
       AVG(DATEDIFF(o.DeliveryDate, o.OrderDate)) AS AvgDeliveryTime
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
JOIN Products p ON od.ProductID = p.ProductID
WHERE o.DeliveryDate IS NOT NULL
GROUP BY p.ProductID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductID	ProductName	AvgDeliveryTime
27	Product 27	46.0000
6	Product 6	-0.5000
10	Product 10	-1.5714
47	Product 47	-100.0000
34	Product 34	7.0000
45	Product 45	37.0000
40	Product 40	37.0000
20	Product 20	7.0000
39	Product 39	7.0000
4	Product 4	5.2000
7	Product 7	5.1667
3	Product 3	4.8333
2	Product 2	5.5000
8	Product 8	5.8333
11	Product 11	4.8333
5	Product 5	5.1667
13	Product 13	5.3333
12	Product 12	5.2000
9	Product 9	5.2000

64.

```sql
-- 64. Determine the day with the highest number of orders
USE FlipkartSales;
SELECT OrderDate, COUNT(*) AS TotalOrders
FROM Orders
GROUP BY OrderDate
ORDER BY TotalOrders DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

OrderDate	TotalOrders
2024-03-23	1

65.

```
1   -- 65. Find repeat customers (placed more than 1 order in the same month)
2   USE FlipkartSales;
3   SELECT CustomerID, OrderMonth, COUNT(*) AS Orders
4   FROM (
5     SELECT CustomerID, DATE_FORMAT(OrderDate, '%Y-%m') AS OrderMonth
6     FROM Orders
7   ) AS MonthlyOrders
8   GROUP BY CustomerID, OrderMonth
9   HAVING COUNT(*) > 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	OrderMonth	Orders
17	2023-07	2

➢ SECTION 9:

66.

```
1   -- 66. Identify products frequently returned and suggest for review
2   USE FlipkartSales;
3   SELECT p.ProductID, p.ProductName, COUNT(r.ReturnID) AS ReturnCount
4   FROM Returns r
5   JOIN Orders o ON r.OrderID = o.OrderID
6   JOIN OrderDetails od ON o.OrderID = od.OrderID
7   JOIN Products p ON od.ProductID = p.ProductID
8   GROUP BY p.ProductID
9   HAVING COUNT(r.ReturnID) > 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductID	ProductName	ReturnCount
4	Product 4	6
7	Product 7	6
3	Product 3	8
10	Product 10	7
2	Product 2	7
8	Product 8	6
11	Product 11	6
5	Product 5	7
13	Product 13	6
1	Product 1	7
15	Product 15	6
14	Product 14	6
6	Product 6	6

67.

```sql
-- 67. Find customers who have not ordered in the last 6 months (churn)
USE FlipkartSales;
SELECT * FROM Customers
WHERE CustomerID NOT IN (
  SELECT DISTINCT CustomerID FROM Orders
  WHERE OrderDate >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH)
);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Conte

CustomerID	CustomerName	Email	City	Region	JoinDate
1	Allison Hill	jillrhodes@miller.com	New Carolyn	North	2021-02-21
2	Michelle Miles	shaneramirez@gmail.com	Lake Roberto	East	2022-05-17
3	Daniel Gallagher	jamesmichael@munoz-roman.info	South Christianport	South	2021-10-13
4	Lisa Hensley	williamsjeremy@galloway-wyatt.com	Ramirezstad	North	2021-06-28
5	Henry Santiago	amandasanchez@gray-mayo.net	Lake Mark	West	2021-03-07
6	Lisa Johnson	keithgibbs@hotmail.com	South William	North	2022-04-19
7	Michael Green	josecoleman@yahoo.com	West Brooke	East	2021-11-30
8	Susan Lopez	martinrodriguez@bishop.com	Lake Michael	South	2021-08-14
9	James Reed	jimmyfox@benson.com	East Jason	West	2021-09-01
10	Jessica Young	josephhall@martin.com	New Kelly	North	2022-06-10
11	Brenda Butler	brendabutler@example.com	Port Henry	South	2021-07-15
12	Franklin Garcia	franklin.garcia@example.com	New Annie	North	2022-02-03
13	Karen Brooks	karenbrooks@example.com	Lake Jose	East	2020-12-09
14	Lawrence Foster	lawrencefoster@example.com	South Kimberly	South	2021-04-28
15	Patricia Russell	patriciarussell@example.com	East Sara	West	2022-01-18
16	Deborah Howard	deborahhoward@example.com	North Kenneth	East	2021-03-12
17	Nicholas Patter...	nicholasp@example.com	Lake Janet	North	2022-06-01
18	Amy Griffin	amygriffin@example.com	West Robert	South	2020-11-25
19	Steven Sanders	stevensanders@example.com	Port Ashley	West	2021-09-20
20	Rachel James	racheljames@example.com	Lake Brenda	East	2022-05-07
21	Albert Cox	albertcox@example.com	New Tammy	North	2021-10-05
22	Catherine Bryant	catherinebryant@example.com	West Karen	South	2020-08-14

68.

```sql
-- 68. Calculate conversion rate by comparing number of visits vs purchases
USE FlipkartSales;
SELECT DATE_FORMAT(OrderDate, '%Y-%m') AS Month,
       SUM(od.SellingPrice * od.Quantity) AS Revenue
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
WHERE OrderDate >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)
GROUP BY Month
ORDER BY Month;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Month	Revenue

69.

```sql
-- 69. Analyze sales drop by category after return spikes
-- Compare month-over-month revenue and returns
USE FlipkartSales;
WITH MonthlyStats AS (
  SELECT
    c.CategoryName,
    DATE_FORMAT(o.OrderDate, '%Y-%m') AS Month,
    SUM(od.SellingPrice * od.Quantity) AS Revenue,
    COUNT(r.ReturnID) AS Returns
  FROM Orders o
  JOIN OrderDetails od ON o.OrderID = od.OrderID
  JOIN Products p ON od.ProductID = p.ProductID
  JOIN Categories c ON p.CategoryID = c.CategoryID
  LEFT JOIN Returns r ON o.OrderID = r.OrderID
  GROUP BY c.CategoryName, Month
)
SELECT * FROM MonthlyStats;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CategoryName	Month	Revenue	Returns
Clothing	2024-03	21561.59	0
Books	2024-03	41211.72	0
Clothing	2024-05	31123.08	1
Books	2024-01	10019.04	0
Sports	2024-04	28242.76	0
Sports	2024-06	3322.53	1
Books	2024-06	5393.36	1
Books	2024-05	28856.20	0
Electronics	2023-06	10092.00	4
Books	2023-06	24190.00	7
Clothing	2023-06	20243.00	4
Home	2023-06	8797.00	4
Sports	2023-06	999.00	1
Books	2023-07	50873.00	14
Home	2023-07	18093.00	6
Electronics	2023-07	11593.00	6
Clothing	2023-07	46439.00	5
Sports	2023-07	7992.00	2
Books	2023-08	46779.00	14
Home	2023-08	16087.00	7
Clothing	2023-08	38338.00	6
Sports	2023-08	5994.00	2
Electronics	2023-08	4997.00	4
Electronics	2023-09	2699.00	2
Books	2023-09	23393.00	4
Home	2023-09	6996.00	3
Clothing	2023-09	17896.00	2
Sports	2023-09	999.00	1

70.

```sql
-- 70. Recommend top 3 categories per region based on total revenue
USE FlipkartSales;
WITH RegionalRevenue AS (
  SELECT
    c.CategoryName,
    cu.Region,
    SUM(od.SellingPrice * od.Quantity) AS Revenue
  FROM Orders o
  JOIN OrderDetails od ON o.OrderID = od.OrderID
  JOIN Products p ON od.ProductID = p.ProductID
  JOIN Categories c ON p.CategoryID = c.CategoryID
  JOIN Customers cu ON o.CustomerID = cu.CustomerID
  GROUP BY c.CategoryName, cu.Region
),
RankedCategories AS (
  SELECT *,
         RANK() OVER (PARTITION BY Region ORDER BY Revenue DESC) AS rnk
  FROM RegionalRevenue
)
SELECT *
FROM RankedCategories
WHERE rnk <= 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CategoryName	Region	Revenue	rnk
Books	East	21685.36	1
Clothing	East	20893.56	2
Home	East	12495.00	3
Books	North	120846.92	1
Clothing	North	82649.03	2
Sports	North	36234.76	3
Clothing	South	58467.08	1
Books	South	47980.00	2
Home	South	17488.00	3
Books	West	31307.04	1
Electronics	West	8698.00	2
Home	West	6849.00	3

Conclusion

The Flipkart Sales Analysis project provides a comprehensive understanding of how data can be used to drive business decisions in the e-commerce domain. By leveraging a well-structured relational database consisting of seven interrelated tables—Customers, Categories, Products, Suppliers, Orders, OrderDetails, and Returns—the project simulates real-world retail operations in a scalable and analyzable format.

Through the use of SQL queries, the project explored over 70 business-focused questions, covering areas such as customer behavior, product performance, order trends, return patterns, and regional contributions. These queries employed advanced SQL concepts like joins, aggregates, subqueries, window functions, views, and common table expressions (CTEs), showcasing the depth of analysis achievable through structured data.

Overall, this project demonstrates how SQL-based analytics can transform raw transactional data into meaningful insights that support strategic decision-making in areas like inventory planning, marketing optimization, and customer engagement. It reinforces the importance of clean, normalized database design and efficient querying in the field of data analytics and business intelligence.

I would like to extend my heartfelt gratitude to our mentors, Mr. Aman Chauhan and Mr. Vijay Narayan Singh, for their constant guidance, encouragement, and valuable feedback throughout the course of this project. Their support played a crucial role in helping me understand the core objectives of data analytics and successfully complete this internship project.

Submitted by:
TANVI AGGARWAL
BCA 2ND YEAR 4TH SEM
Tecnia Institute of Advanced Studies ,GGSIPU

End of Report